Merge Linked Lists

AlgoExpert · Hard · Linked Lists

The problem

- Merge two sorted lists in place; return the head of the merged list.

Intuition

- Walk both sorted lists, splicing the smaller node in front each time, without new nodes.

Brute force vs optimal

- Copy into an array, sort, rebuild → O((n+m) log)
- Iterative merge with a dummy head → O(n+m), O(1)
- Recursive merge → O(n+m) stack

Approach

- 1. Keep pointers into both lists and a previous pointer
- 2. Splice the smaller node
- 3. Attach any leftover tail
- 4. Return the smaller head

Worked example

list 1



list 2



merged



Take 1 from the second list

Worked example

list 1



list 2



merged



Take 2 from the first list

Worked example

list 1



list 2



merged



Take 4 from the second list

Worked example

list 1



list 2



merged



Take 6 from the first list

Worked example

list 1



list 2



merged



Take 8 from the first list

Worked example

merged



Attach the remainder

Java solution

```java
public class MergeLinkedLists {
    public static LinkedListNode mergeLinkedLists(LinkedListNode a, LinkedListNode b) {
        LinkedListNode dummy = new LinkedListNode(0), t = dummy;
        while (a != null && b != null) {
            if (a.value <= b.value) { t.next = a; a = a.next; } else { t.next = b; b = b.next; }
            t = t.next;
        }
        t.next = a != null ? a : b;
        return dummy.next;
    }
}
```

Complexity

- Time: O(n + m)
- Space: O(1)

Edge cases & pitfalls

- Empty list
- Use ≤ for stability
- Attach the leftover directly

Interview tips & follow-ups

- Follow-up: merge k lists (heap)

Recap

- Walk both sorted lists, splicing the smaller node in front each time, without new nodes.
- Complexity: O(n + m)